

Proposal: StrictDoc–KiCad Integration

Bidirectional Requirements Management, Native Traceability, and Continuous Validation

Author: Steve Lazzeri, Stanislav Pankevich

Version: 1.1 Draft

Date: August 16, 2026

1. Introduction

Software and safety-critical industries widely use requirements engineering to improve traceability, verification, and overall project quality. This proposal pursues two goals:

1. Make requirements a first-class part of KiCad, so hardware engineers can manage, trace, and validate them without leaving their design environment.
2. Integrate KiCad with StrictDoc, one of several capable, open-source, text-based requirements management tools.

The implementation covers three phases:

1. **Plugin (Minimum Viable Product)** – Validate engineering workflows and bidirectional synchronization while gathering early community feedback.
2. **CI/CD (Minimum Viable Product)** – Introduce automated validation, traceability verification, and engineering reporting.
3. **Native KiCad Integration** – Investigate long-term native support for requirements management, traceability, and validation within KiCad.

What is KiCad?

[KiCad](#) is an open-source suite of tools for designing electronic hardware, covering schematic capture, PCB layout, and 3D visualization, with built-in electrical rule checking (ERC) and design rule checking (DRC). KiCad stores projects and their design data as text-based files (such as `.kicad_pro`, `.kicad_sch`, and `.kicad_pcb`), which makes it practical to synchronize KiCad projects with an external tool like StrictDoc.

What is StrictDoc?

[StrictDoc](#) is an open-source tool for managing requirements as plain text, rather than in a binary or database format, so engineers can version and diff documents in Git like source code. StrictDoc supports several text markup languages, most notably Markdown and SDoc. From these text inputs, StrictDoc builds a graph of the requirements to track relationships such as traceability links, then exports that graph to formats including HTML, ReqIF, and JSON.

2. Vision

This proposal is built on two goals:

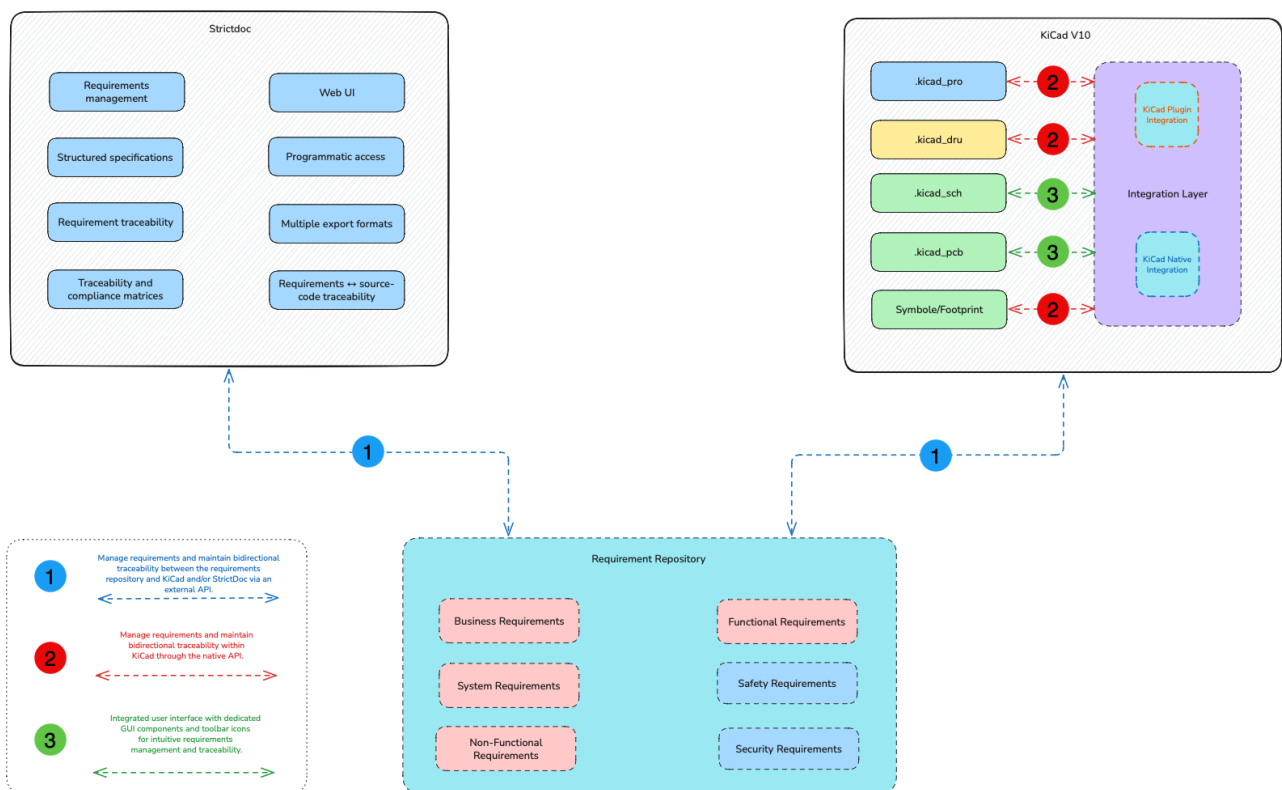
1. Make requirements first-class objects within KiCad itself, so engineers can associate them with hardware design elements, review and verify them, and trace them throughout the hardware development lifecycle without leaving KiCad.
2. Implement an integration with StrictDoc, a capable, text-based, open-source requirements tool. During the implementation, look for opportunities to generalize the integration to support any open-source or commercial tool.

New `.kicad_req` format

This proposal introduces `.kicad_req`, a new project metadata file that stores synchronized requirements, versions, and traceability information independently of KiCad design files. It enables offline use, change detection, and synchronization with StrictDoc and similar tools.

The integration should support:

- Block diagrams
- Schematic symbols
- PCB objects and nets
- Design rules
- Verification activities
- Automated CI/CD validation



3. Expected Capabilities

The proposed integration should provide the following core engineering capabilities:

Requirements Engineering

- Import and synchronize requirements between KiCad and StrictDoc.
- Manage and browse requirements within KiCad.
- Maintain requirement hierarchies and properties.

Traceability

- Allocate requirements to hardware design elements.
- Support bidirectional navigation between requirements and hardware objects.
- Display implementation and verification status.

Verification & Validation

- Validate requirement synchronization and project consistency.
- Detect broken references, missing implementations, and missing verification activities.
- Identify duplicate requirement IDs and orphaned design objects.

Reporting

- Generate traceability reports.
- Generate verification and coverage reports.
- Generate CI/CD validation summaries.

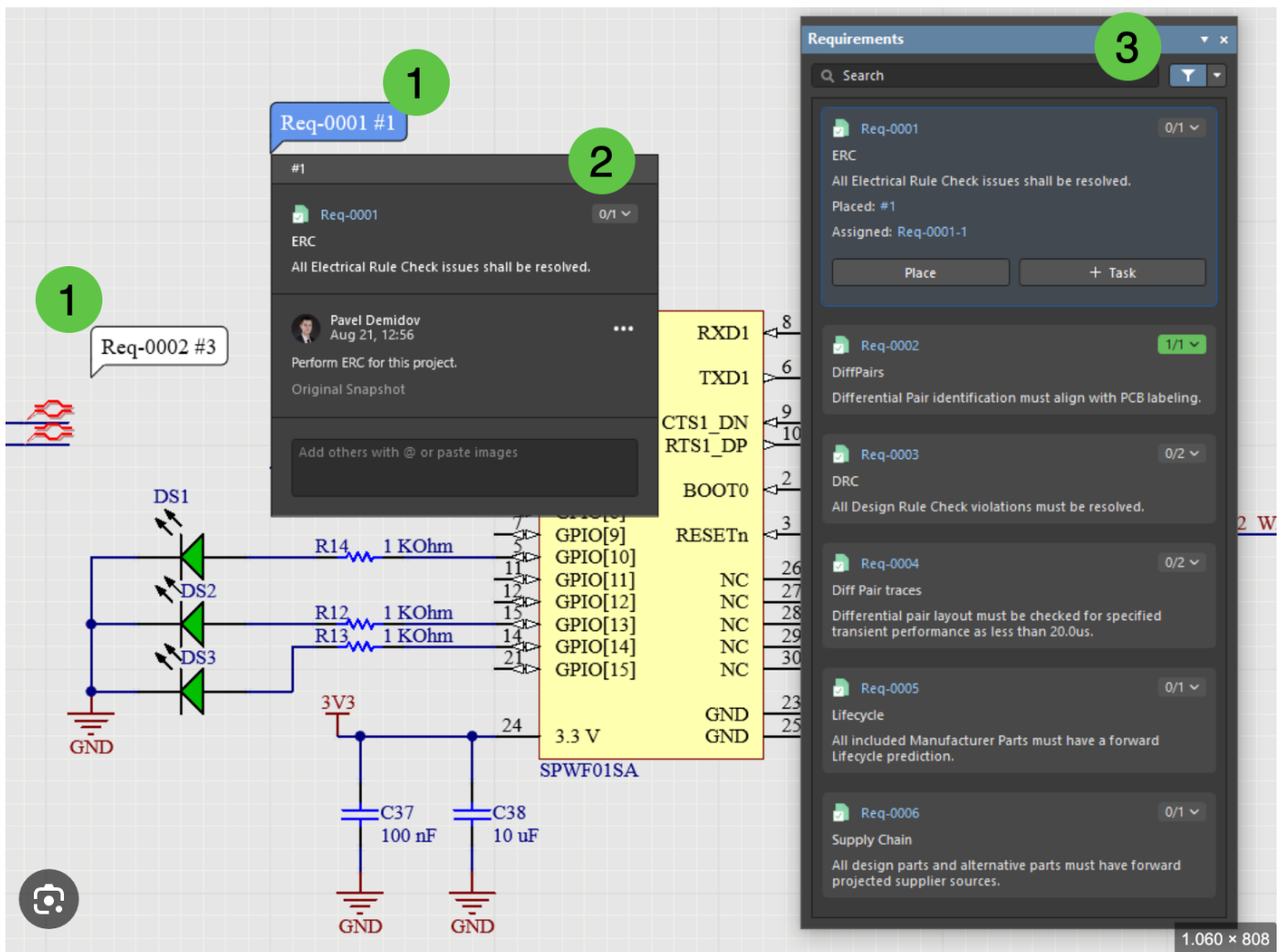
Examples from Altium

The following images and screenshots from Altium illustrate the proposed requirements management and traceability workflow. Altium is another tool for designing electronic hardware. It already has requirements features, which show what this kind of workflow could look like in KiCad.

The image is a composite of three screenshots from Altium Designer illustrating a requirements management workflow:

- Top Screenshot: 'All Requirements' Table**
 - A red circle '1' highlights the 'All Requirements' tab in the top-left pane.
 - The table lists requirements with columns: Identifier, Text, Applicable Block, Design References, V&V S..., V&V Activities, and V&V Rules.

Identifier	Text	Applicable Block	Design References	V&V S...	V&V Activities	V&V Rules
Req-0001	All Electrical Rule Check issues shall be resolved	WiFi_Connectivity	Connector_WiFi.SchDoc	1 / 1	Compliance	0 / 0
Req-0002	Differential Pair Identification must align with PCB labeling.	WiFi_Connectivity	Connector_WiFi.SchDoc Connector_WiFi.SchDoc	1 / 1	Compliance	0 / 0
Req-0003	All Design Rule Check violations must be resolved.	WiFi_Connectivity		0 / 2	Compliance Review	0 / 0
- Bottom Left Screenshot: PCB Schematic**
 - A red circle '2' highlights a callout for 'Req-0002 #2' pointing to a differential pair on the PCB.
 - Another red circle '2' highlights a callout for 'Req-0001 #1' pointing to a component on the PCB.
- Bottom Right Screenshot: Requirement Card**
 - A red circle '3' highlights the 'Req-0001' card, which shows the requirement text and a 'Place' button.
 - A red circle '4' highlights the 'Req-0001' card in the 'Requirements' panel on the right, showing the requirement text, assigned ID, and a 'Task' button.



4. Roadmap

Phase 1 – Plugin (Minimum Viable Product)

Develop a prototype to validate engineering workflows without requiring modifications to the KiCad core, using StrictDoc as the first requirements tool it should integrate with.

- Prototype requirements synchronization between KiCad and StrictDoc.
- Validate core engineering workflows and traceability.
- Gather community feedback to guide future development.

Phase 2 – CI/CD (Minimum Viable Product)

Extend the plugin with automated validation and traceability verification.

- Automate requirement and project validation.
- Verify end-to-end traceability throughout the design lifecycle.
- Generate engineering and CI/CD validation reports.

Phase 3 – Native KiCad Integration

Investigate long-term native support for requirements engineering within KiCad, so requirements become fully first-class objects rather than data synchronized from an external tool.

- Integrate native requirements management and traceability.
- Provide public APIs for extensions and automation.

- Deliver an enhanced, fully integrated user experience.

5. Example Use Cases

Example 1: Requirement Allocation

A hardware engineer imports project requirements from StrictDoc into KiCad, then allocates requirement `REQ-PWR-001` to the power supply block diagram and links it to the regulator schematic symbol, PCB footprint, power net, and associated design rule.

When the engineer selects any linked object, KiCad displays the requirement properties, implementation status, verification information, and a direct link to the corresponding requirement in StrictDoc. This lets engineers quickly see which design elements implement a requirement and verify its traceability throughout the hardware design.

Example 2: Requirement Change and Validation

A system engineer updates `REQ-PWR-001` in StrictDoc. The synchronization engine propagates the changes to the KiCad project and highlights all affected design objects for review.

Before release, the CI/CD pipeline automatically validates the project, including StrictDoc documents, requirement synchronization, traceability, project integrity, ERC, and DRC, and generates a comprehensive validation report.

6. Technical Investigation

The technical investigation should evaluate the feasibility of integrating StrictDoc and KiCad, with particular attention to:

- Plugin architecture and extensibility.
- Native KiCad APIs and integration points.
- Requirements synchronization and the underlying data model.
- User interface design and engineering workflows.
- Performance, scalability, and long-term maintainability.
- Compatibility with future KiCad releases.

7. Success Criteria

The integration succeeds if it achieves the following objectives:

Functional

- Manage and work with requirements within KiCad.
- Synchronize requirements between KiCad and StrictDoc, the first supported requirements tool.
- Support bidirectional navigation between requirements and hardware objects.
- Enable requirement allocation throughout the hardware design.

Verification

- Provide automated validation and traceability verification.
- Integrate with CI/CD workflows.
- Generate comprehensive traceability and validation reports.

Quality

- Scale to support large and complex hardware projects.
- Maintain a modular, extensible, and maintainable architecture.

8. Open Source Collaboration

The project will develop the integration as an open source project under the MIT License, and will hold design discussions, implementation, issue tracking, and documentation in public repositories.

The success of this initiative depends on collaboration between the StrictDoc and KiCad communities. Developers, maintainers, and users will work together to define the architecture, refine engineering workflows, and guide the project's long-term direction.

Contributions of all kinds are welcome. Community members can contribute through:

- Design discussions, technical proposals, and code contributions.
- Testing, documentation, bug reports, and feature requests.
- Code reviews, user feedback, mentoring, and community outreach.

An open, collaborative development process will build a maintainable, extensible foundation for hardware requirements engineering within the KiCad ecosystem.

9. Next Steps

Next, the project will engage the StrictDoc and KiCad communities and refine the proposed architecture and implementation roadmap.

The initial activities are:

1. Review and refine this proposal.
2. Publish the proposal for community discussion.
3. Engage with the KiCad maintainers and the StrictDoc community.
4. Review the proposed engineering workflows and architecture.
5. Evaluate candidate implementation approaches.
6. Estimate the development effort.
7. Define the scope of the Plugin (Minimum Viable Product).
8. Establish milestones and begin development of the Plugin MVP.

10. Conclusion

This proposal presents an incremental approach to making requirements a first-class part of KiCad, with StrictDoc as the first text-based requirements management tool the integration supports.

The project starts with a lightweight plugin built around StrictDoc and moves toward native KiCad integration. Early phases test the plugin on real projects and build collaboration between the StrictDoc and KiCad communities. Once that groundwork is solid, the project can add support for other requirements tools.